

Log-Sum-Exp, Softmax, and Likelihood Underflow in the EM Engine

Context: Notes from implementing `compute_prior_from_beta`, `compute_ylik`, and the E-step in the C++ EM engine.

1. Why beta is a flat vector

Conceptually, `beta` is an $S \times (R-1)$ matrix where: - S = number of covariates (columns of x) - R = number of latent classes (`nclass`)

Newton-Raphson operates on a flat parameter vector: - **Gradient:** length $S(R-1)$ - **Hessian:** $S(R-1) \times S(R-1)$ - **Update:** $\beta_{\text{new}} = \beta_{\text{old}} - H^{-1}g$

So `Params.beta` is stored as `Eigen::VectorXd` of length $S(R-1)$. Inside `compute_prior_from_beta` you map it back with `Eigen::Map`:

```
Eigen::Map<const Eigen::MatrixXd> beta_mat(beta.data(), S, R - 1);
```

This is zero-overhead and avoids constant reshaping at call sites.

2. Stabilized softmax (the “subtract max” trick)

The multinomial logit prior is:

$$P(\text{class} = r \mid x_i) = \frac{\exp(\eta_{ir})}{1 + \sum_{s=1}^{R-1} \exp(\eta_{is})}$$

with linear predictors $\eta_{ir} = x_i \cdot \beta_{\cdot r}$ and baseline class R having implicit $\eta = 0$.

Problem: If any η is large (e.g. > 700), $\exp(\eta)$ overflows to `inf` in double precision.

Solution: Subtract $M = \max(0, \eta_1, \dots, \eta_{R-1})$ before exponentiating. This is equivalent to multiplying numerator and denominator by $\exp(-M)$:

$$P_{ir} = \frac{\exp(\eta_{ir} - M)}{\exp(-M) + \sum_{s=1}^{R-1} \exp(\eta_{is} - M)}$$

- The largest exponent becomes $\exp(0) = 1$ — nothing overflows.
 - The ratio is unchanged because the same factor multiplies numerator and denominator.
 - The baseline class contributes $\exp(-M)$ in the denominator, keeping everything balanced.
-

3. The connection to log-sum-exp

The **log-sum-exp** (LSE) function is:

$$\text{LSE}(z) = \log \left(\sum_j e^{z_j} \right)$$

Its stabilized form is:

$$\text{LSE}(z) = \max(z) + \log\left(\sum_j e^{z_j - \max(z)}\right)$$

Now look at softmax expressed via LSE:

$$P_r = \frac{e^{z_r}}{\sum_j e^{z_j}} = \exp(z_r - \text{LSE}(z))$$

For softmax alone you **do not need the final log**. You just subtract the max, exponentiate, and divide. The “subtract max” step in softmax *is* the stabilization trick from LSE, applied without taking the logarithm at the end.

4. Where raw likelihoods underflow

`compute_ylik` currently multiplies per-item probabilities:

```
ylik(i, r) *= prob_rjk;    // product over j items and categories
```

With many items, this product underflows to 0. Then the E-step divides 0/0.

You never actually need the raw likelihood values. You only ever use them in: 1. **Ratios** (posterior probabilities) 2. **Logarithms** (log-likelihood)

Both can be computed entirely in log-space.

5. Working in log-space: the real use of LSE

E-step via log terms

Define log-terms for each observation i and class r :

$$z_{ir} = \log(\text{prior}_{ir}) + \underbrace{\sum_j \log P(y_{ij} | r)}_{\log\text{-ylik}_{ir}}$$

Then the posterior is a softmax over the classes:

$$\text{posterior}_{ir} = \exp(z_{ir} - \text{LSE}_r(z_{i.}))$$

This is numerically stable because: - You never materialize the tiny probability product. - The LSE computation itself uses the “subtract max” trick.

Log-likelihood

The marginal log-likelihood becomes:

$$\ell = \sum_{i=1}^N \log\left(\sum_r \text{prior}_{ir} \cdot \text{ylik}_{ir}\right) = \sum_{i=1}^N \text{LSE}_r(z_{i.})$$

LSE gives you the log-likelihood **directly**.

6. Recommended implementation

1. Change `compute_ylik` to return log-likelihoods:

```
Eigen::MatrixXd compute_log_ylik(const Data& data, const Params& p, int nclass);
```

Replace `ylik(i, r) *= prob` with `log_ylik(i, r) += std::log(prob)`.

2. Stabilize `compute_prior_from_beta` with the subtract-max trick (as shown in §2).

3. Rewrite `e_step` to operate on log-terms and use LSE:

```
Eigen::MatrixXd e_step(..., const Eigen::MatrixXd& prior, ...) {  
    Eigen::MatrixXd log_ylik = compute_log_ylik(data, p, nclass);  
    // For each i: z = log_prior.row(i).transpose() + log_ylik.row(i).transpose();  
    // LSE = stable_log_sum_exp(z);  
    // posterior.row(i) = (z.array() - LSE).exp();  
}
```

4. Compute the overall log-likelihood by summing the per-observation LSE values.

7. Summary table

Task	Expression	Needs LSE?
<code>compute_prior_from_beta</code>	$\text{Beta}(x_i)$ as a probability vector	No — stabilized softmax (subtract max, exp, divide) is sufficient
E-step (posterior)	$\exp(z_r - \text{LSE}(z))$	Yes — LSE in the denominator
Log-likelihood	$\sum_i \text{LSE}_r(z_{i.})$	Yes — LSE is the exact quantity you need

8. Key insight

The log-sum-exp trick is not an alternative to computing likelihoods. It is the way to **avoid ever forming tiny likelihood products** while still computing exactly the same posteriors and log-likelihoods.